Predictive TCP: Simple Explanation and Review Guide

Course: BCSE308L Computer Networks, VIT (Prof. Sreethar S) **Team:** Antra (24BDS0066) · Anuska Sen (24BCE2219) · Varshini Harikrishna (24BCE0780) **Repository:** https://github.com/Antra1705/CN

This guide explains the project in plain language and shows how to present it in the review: what to say, what to demo, and how to answer the likely questions. All numbers are the final results (10 seeds, compared with TCP Reno).

---

1. The Project in 30 Seconds

Say this first, in the review, before anything else:

> "Normal TCP only slows down *after* the router has already dropped packets. We built **Predictive TCP**, a TCP version that sees congestion **coming**. Every 100 ms it runs a tiny machine-learning model, a decision tree with just 4 rules, that predicts from the sender's own delay measurements whether the router queue will be full **0.5 seconds from now**. If yes, it slows down a little, early. In the NS-3 simulator this gives about **90% fewer packet drops** and **28–36% lower delay** than TCP Reno, while keeping **97–100% of its speed**."

---

2. The Problem (Simple Version)

Think of a road with a toll booth. Many cars (packets) from many drivers (senders) must pass through one narrow toll booth (the *bottleneck link*, 10 Mbps). Cars waiting form a queue. The waiting area has room for only 50 cars (the router's *buffer*). When it is full, new cars are turned away: the packet is **dropped**.

What normal TCP does:

- It keeps speeding up (sending more and more) until a packet is dropped.
- Only then does it slow down (Reno halves its sending window).
- Then it speeds up again, fills the queue again, and loses packets again. This repeats forever: the **saw-tooth** pattern.

Two costs: 1. **Drops:** lost packets must be sent again (*retransmissions*), which wastes bandwidth. 2. **Delay:** while the queue is full, every packet waits in it. That extra waiting is felt in video calls, games and web pages.

---

3. The Idea

As the queue fills, the delay rises before any packet is lost. The sender already measures delay: the *round-trip time* (RTT), the time from sending a packet to getting its acknowledgement. So the sender can

notice "the queue is building up" early.

Why machine learning? A fixed rule ("slow down if RTT doubles") needs hand-picked thresholds. Instead, we let a model **learn** from thousands of examples which delay patterns come before a full queue.

Why a tiny model? TCP runs inside the operating system, for every connection, all the time. A big model would be too slow. Our decision tree has depth 2 and 4 leaves: it is just two `if` checks, exported automatically to C++.

One rule we kept strictly: the model only uses things the **sender can measure itself** (its own RTT). The router's queue length is used only to create the training labels, never as an input.

---

4. How It Works, Step by Step

Step	What happens	Tool
1. Simulate	A network with many senders sharing one 10 Mbps link is simulated with TCP Tahoe, Reno, CUBIC, Vegas and BBR, at 3 congestion levels.	NS-3.48 (C++)
2. Record	Every 100 ms, for every flow: RTT, window size, losses, and the real router queue.	NS-3 trace logging
3. Label	Each moment gets a label from the **real queue 0.5 s later**: Low (< 40% full), Medium (40–80%), High (≥ 80% full or a drop).	Python, pandas
4. Train	A decision tree learns to predict the label from the sender's RTT features. 106,354 samples.	scikit-learn
5. Export	The tree is converted to C++ code automatically and checked: it gives the same answer as Python on all 42,235 test samples.	`export_tree.py`
6. Use	The new TCP class `TcpPredictive` runs the tree every 100 ms and reacts.	NS-3 (C++)
7. Evaluate	Predictive TCP is compared with all 5 TCP versions on 10 random seeds, plus different buffers, link speeds and competition with CUBIC.	NS-3 + Python

The 4 rules the tree learned

RTT ÷ lowest RTT seen	RTT ÷ its 1-second average	Prediction (0.5 s ahead)	What Predictive TCP does
≤ 1.23	(not checked)	**Low**	Grow normally, like Reno
> 1.23	≤ 0.85 (queue emptying fast)	**Medium**	Grow slowly (25% of Reno's speed)
> 1.23	> 0.85	**High**	Cut the window by 20% (at most once per RTT) and stop growing

In words: "If the delay is more than 23% above the empty-network delay and it is not falling quickly, the queue will be full soon, so back off a little now."

- **20% cut instead of 50%:** Predictive TCP cuts early and gently instead of late and hard.
- **Safety net:** if a packet is still lost, the normal Reno halving happens too.

- **Minimum window:** with a window below 4 packets, the model is not used, so the flow never starves itself.

The competition detector (added later)

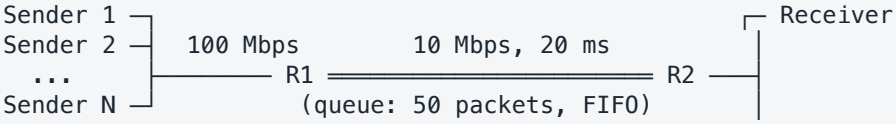
Problem found: when Predictive TCP shares the link with CUBIC, CUBIC keeps the queue full. Predictive TCP keeps backing off while CUBIC takes the bandwidth. At first Predictive TCP got only 23% of the link (a fair share is 50%).

Fix:

- **The signal:** if the lowest RTT over 2 seconds stays above 1.4 × the empty-network RTT, the queue *never* empties. That means an aggressive, loss-based sender is present.
- **The response:** Predictive TCP then behaves like normal Reno. It switches back 5 s after the last evidence.
- **The result:** its share rises from 23% to **40%**.
- **No cost when alone:** when only Predictive TCP flows share the link, the detector never fires.

---

5. The Simulated Network



- **Dumbbell topology:** many senders → router R1 → **bottleneck link** → router R2 → receiver.
- **Empty-network RTT:** 44 ms.
- **Three congestion levels:**

Level	Traffic	Link load
Low	3 on/off TCP flows	~30%
Medium	6 on/off TCP flows	~90%, bursty
High	8 always-on TCP flows + a UDP flow	Full

- **Each run lasts 60 seconds.** Every setting is repeated with 10 random **seeds** (different start times and traffic patterns), and the results are averaged.
- **Train/test split:** seeds 1–3 were used for every choice (training, model selection, tuning). Seeds 4–10 were never used for any decision, so they show the model is not overfitted.

---

6. Results

Main result (10 seeds, compared with TCP Reno)

Metric	Medium load	High load
Packet drops	**–93%**	**–90%**
Retransmissions	**–93%**	**–89%**
Average RTT (delay)	**–28%** (59 vs 82 ms)	**–36%** (65 vs 102 ms)
Goodput (useful speed)	97.4% of Reno	99.6% of Reno
Fairness (Jain index, 1 = perfectly fair)	0.99 (Reno 0.99)	0.95 (Reno 1.00)

What makes these results trustworthy:

- **Every seed:** fewer drops and lower delay than Tahoe, Reno, CUBIC and BBR in all 10 seeds at both load levels.
- **Unseen seeds 4–10:** the same size of gain (drops –94% / –90%, RTT –29% / –36%).
- **Statistically significant:** every drop and delay improvement over Reno and CUBIC has $p < 0.0001$ (paired t-test, confirmed by Wilcoxon and sign tests).
- **All 9 success criteria** from our PRD are met (for example "drops at least 30% lower than Reno", "goodput at least 90% of Reno", "fairness at least 0.9").

Against the other TCP versions

	Medium load	High load
vs CUBIC	RTT –31%, far fewer drops	RTT –40%, far fewer drops
vs BBR	drops –98%, RTT –34%	drops –98%, RTT –40%, fairer (0.95 vs 0.85)
vs Vegas	Vegas has fewer drops; we have lower RTT (59 vs 65 ms)	lower RTT in all 10 seeds (65 vs 107 ms); fewer drops in 7 of 10 seeds

The machine-learning model

Model	Macro-F1 (3 classes)	Recall for "High"
Decision tree (ours)	**0.60**	**0.86**
Logistic regression	0.54	0.62
Simple loss-counting rule	0.36	0.50

- **Best of the three:** the tree beats the other two on unseen seeds.
- **Better than guessing:** compared with always guessing "High", it scores MCC 0.56 vs 0.

What each part contributes (ablation, high load)

- **Without the 20% early cut:** drops rise from 164 to 276 and delay from 65 to 107 ms. The early cut is the main mechanism.
- **Without the competition detector:** the share against CUBIC falls from 40% to 23%.

---

7. Honest Weaknesses (say these yourself)

Examiners respect a team that knows its limits. Mention these before you are asked:

1. **Sharing with CUBIC:** 40% of the bandwidth instead of a fair 50%. The detector helps (23% → 40%), but does not fully solve it.
2. **Fast or long links are unfair:** at 50 Mbps, or with 100 ms delay, a few flows take most of the bandwidth (Jain as low as 0.37). Drops are still lower than Reno's in every case.
    - **Cause:** *delayed ACKs* make the RTT of small windows look higher than it is, so small flows keep backing off.
    - **We tested a fix:** it makes the flows fair but loses about half of the main gain, so we kept the current model and documented the trade-off.
3. **False alarms with big buffers:** with a 100-packet buffer at high load, the competition detector fires wrongly. The gain then shrinks (drops –40% instead of –86%). The detector can be switched off.
4. **The ML model's overall score (macro-F1 0.60) is below our 0.70 target.** At high load almost every moment is "High", so there is little left to predict there. The model matters most at medium load, where it beats the trivial guess clearly.
5. **Simulation only:** one topology and one traffic mix, no real hardware. A small cost: about 2.6% less speed at medium load.

---

8. How to Present It in the Review

Suggested flow (about 10 minutes)

Time	Part	Who (suggested)	What to show
1 min	The problem	Speaker 1	Section 2: toll-booth analogy, saw-tooth, drops + delay
1 min	The idea	Speaker 1	Section 3: RTT rises before a drop; tiny model
2 min	How it works	Speaker 2	Section 4 table + the 4 tree rules
3 min	Live demo	Speaker 2	The app (see below)
2 min	Results	Speaker 3	Main result table, "10/10 seeds", $p < 0.0001$
1 min	Weaknesses + conclusion	Speaker 3	Section 7, then the 30-second summary again

Divide the speaker roles among Antra, Anuska and Varshini as you prefer. Each person should still be able to answer questions on every part.

Live demo script

Before the review:

- Open a terminal in the project folder and run `./app/run_app.sh`. The app opens at http://localhost:8501.
- Run the default experiment once so it is cached.
- Keep `results/RESULTS.md` open as a backup in case the app fails.

In the review: 1. **Compare variants, High load, seed 4** (a seed never used in training). Press **Run simulation**. - *Say:* "Each run is a real 60-second NS-3 simulation. Look at the queue chart: Reno and CUBIC keep the queue full, Predictive TCP keeps it short. That is why delay and drops are lower." 2. **Add BBR and Vegas** to the variants and run again. - *Say:* "BBR floods the queue and loses many packets. Vegas is also delay-based and has few drops, but higher delay than us at high load." 3. **Change the queue size to 100 packets.** - *Say:* "With a bigger buffer, normal TCP fills it even more (bufferbloat), so our delay advantage grows." 4. **Mode → Share with CUBIC**, Predictive TCP, detector on. Then switch the detector off in *Predictive TCP parameters* and run again. - *Say:* "This is our main weakness. Without the detector we get about a quarter of the link; with it, about 40%." 5. **How it works tab:** show the 4 decision-tree rules and the generated C++ code. - *Say:* "This is the entire model. It runs inside the TCP sender."

---

9. Likely Questions and Good Answers

Q1. What is new in your project? Delay-based TCP (Vegas) already exists. Vegas uses a fixed formula on the *current* delay. We use a model **trained on data** to predict congestion **0.5 s in the future**, on top of Reno. Against Vegas we have lower delay at high load in all 10 seeds, and fairer sharing (0.95 vs 0.87).

Q2. Why a decision tree and not a neural network?

- **Speed:** it must run inside TCP every 100 ms for every connection, so it has to be tiny and fast. Our tree is two `if` checks.
- **Explainability:** we can read and explain every rule.
- **Accuracy:** it was already the best of the models we tried (macro-F1 0.60 vs 0.54 for logistic regression).
- **Size:** deeper trees did not score better in cross-validation.

Q3. Why predict 0.5 seconds ahead?

- **Short enough:** the delay signal is still reliable.
- **Long enough:** a reaction still has time to take effect. 0.5 s is about 11 round trips.
- **Tested:** we compiled 5 candidate models (0.2, 0.5 and 1.0 s horizons, different feature sets) into the simulator. The winner, delay features with a 0.5 s horizon, cut drops by about 88% vs Reno; the next best managed about 64%.

Q4. How did you get the labels? In the simulator we can see the real router queue. Each sample is labelled with the queue occupancy **0.5 s later**: Low below 40% full, Medium 40–80%, High at 80% or more, or if a packet was dropped. The sender never sees the queue; it only uses it during training.

Q5. How do you know you did not overfit?

- **Separate seeds:** every choice (training, model selection, tuning) used seeds 1–3 only.
- **Unseen results:** on seeds 4–10, which were never used for any decision, the gains are the same size.
- **Exact export:** the C++ export gives exactly the same predictions as Python on all 42,235 test samples.

Q6. What is a seed, and why 10? A seed sets the random parts of a run: start times and on/off traffic patterns. Ten seeds means 10 different but repeatable scenarios. With 10 we can run proper statistical tests; every main gain has $p < 0.0001$.

Q7. Why did goodput go down slightly? Backing off early leaves the link idle for a moment now and then. The cost is small: 97.4% of Reno's goodput at medium load and 99.6% at high load. In return there are 90% fewer drops and up to 36% less delay. Alone on an empty link it reaches 9.7 Mbps (Reno 9.0).

Q8. What is the Jain fairness index? A number from 0 to 1 showing how equally flows share the link; 1 means every flow gets exactly the same. Ours is 0.99 at medium load and 0.95 at high load (target ≥ 0.9).

Q9. Why do you lose against CUBIC? CUBIC only slows down after a loss, so it keeps the queue full. Predictive TCP sees the full queue as "congestion coming" and keeps backing off, so CUBIC takes the space. Our competition detector notices that the queue never empties and switches to Reno behaviour: 23% → 40%. Every delay-based TCP, Vegas included, has this problem (Vegas gets 34%).

Q10. Why does it become unfair on a 50 Mbps link?

- **Cause:** with *delayed ACKs*, the receiver acknowledges every second packet. A flow with a very small window often waits for that second packet, so its measured RTT looks higher than it is. The model reads this as congestion and the small flow keeps shrinking.
- **The fix we tested:** fixing the signal makes the flows fair, but loses about half the main gain.
- **Our decision:** we kept the main result and documented the trade-off openly.

Q11. Why NS-3 and not real computers?

- **Real queue:** NS-3 shows the true router queue at every moment, which we need for labels.
- **Repeatable:** every run is exactly repeatable.
- **Fair comparison:** all 6 TCP versions run in identical conditions.

On real hardware we could not see the queue or repeat a run exactly.

Q12. What is BBR, and why does it do so badly here? BBR (Google) estimates bandwidth and delay instead of reacting to loss. Version 1, the one in NS-3, is known to overfill shallow buffers. With a 50-packet buffer it loses about 5× more packets than Reno.

Q13. What does β = 0.8 mean, and how was it chosen? On a "High" prediction the window is multiplied by 0.8, a 20% cut. We tried 18 combinations of β and minimum window on the training seeds. Halving (β = 0.5) gave even fewer drops at medium load, but under-used an empty link, so it failed our single-flow speed check.

Q14. How can this be reproduced?

- **Full run:** `./run_all.sh` re-runs everything in about 28 minutes and gives byte-identical results each time.
- **Without NS-3:** `./run_all.sh --reference` rebuilds every table and graph from the saved simulation data in about 1 minute, on any system.

Q15. What would you do next?

- **Fairness:** give each flow a target share of the queue (like FAST TCP or Copa) instead of the same cut for every flow. This should fix the unfairness on fast links.

- **Delayed ACKs:** handle them in the RTT signal.
- **Wider tests:** more topologies and traffic mixes, and eventually a Linux kernel module.

---

10. Key Terms (one line each)

Term	Meaning
Bottleneck	The slowest link on the path; packets queue up in front of it
Buffer / queue	Memory in the router where waiting packets are stored (50 packets here)
Drop	A packet thrown away because the queue is full
cwnd (congestion window)	How many packets TCP may have "in flight" at once; bigger = faster
RTT (round-trip time)	Time from sending a packet to receiving its ACK; rises when the queue fills
Goodput	Useful data delivered per second (excludes retransmissions)
Retransmission	Sending a lost packet again
BDP (bandwidth-delay product)	Bandwidth × RTT: how much data fills the pipe (≈ 38 packets here)
Bufferbloat	Large buffers that loss-based TCP keeps full, causing high delay
Delayed ACK	The receiver acknowledges every second packet, not every packet
Jain fairness index	0–1 measure of how equally flows share the link
Seed	Number that fixes the random parts of a simulation run
Macro-F1	Average prediction quality over the three classes (Low, Medium, High)
p-value	Probability of seeing a difference this big by chance; < 0.05 = significant
Ablation	Removing one part of the design to see how much it matters

---

11. Numbers to Remember

What	Number
Drop reduction vs Reno	~90% (–93% medium, –90% high)
Delay reduction vs Reno	28% medium, 36% high
Goodput kept	97–100% of Reno
Model size	depth 2, 4 leaves (2 `if` checks)
Prediction horizon	0.5 s ahead, checked every 100 ms
Dataset	106,354 samples
Reaction	cut window by 20% ($\beta = 0.8$)
Seeds	10 (1–3 for choices, 4–10 unseen)
Significance	$p < 0.0001$ for all main gains
Share vs CUBIC	40% (23% without the detector)
Network	10 Mbps, 20 ms, 50-packet queue, base RTT 44 ms